

Artificial Intelligence in Pla...

★ Member-only story

Before You Vibe Code, Write These 7 Files

The reason AI keeps building the wrong thing — and how to fix it before you write a single line of code.



Ruqaiya Beguwala

Follow

4 min read · Mar 29, 2026



45



Non-members, please read [here](#).



Image source: UX Collective

Most people start vibe coding by opening a chat window and typing something like *“build me a task management app.”*

Then they spend the next three hours fixing things that were never supposed to be built in the first place.

The problem isn't the AI. The problem is that you handed it a napkin sketch and expected a blueprint.

Here's a better way.

 Library Profile Stories Stats Following Python in Plain En... Generative AI AI Quick Tips Dev Genius Top Python Librari... Realworld AI Use C... Level Up Coding AI Mind The Useful Tech All About ChatGPT More

Documentation First. Code Second.

Before writing a single line of code, create six canonical markdown files. These serve as your project's knowledge base. The AI reads them, knows what "done" looks like, and stops guessing.

The fewer guesses the AI makes, the better the results.

The 6 Files (+ 1 Bonus)

1. PRD.md — What You're Building

Your product requirements document. Write what the app does, every feature it includes, and just as importantly what is **out of scope**. If you don't specify the boundaries, the AI will create them.

Prompt to create it:

“Act as a product manager. Based on this idea: [your idea], write a PRD.md that covers the purpose, core features, out-of-scope items, and success criteria. Keep it concise and clear.”

2. APP_FLOW.md — Every Page, Every Path

Create a map of every screen and user journey. How does a user move from the landing page to the checkout? What happens after they have logged out? If navigation is not documented, the AI guesses, and it guesses incorrectly.

Prompt to create it:

“Write an APP_FLOW.md for this app: [brief description]. List every page, the user paths between them, and any conditional flows (logged in vs. logged out, empty states, errors).”

3. TECH_STACK.md — Exact Versions Only

Talk it out with your AI tool what is the best tech stack to use. No vague instructions like “*use React*”. Write React 18.2.0. Write Node 20.11.0. Write Tailwind 3.4.1. Discuss what’s needed and update it in md file.

Prompt to create it:

“Write a TECH_STACK.md for a [type] app using [your preferred stack]. Include exact versions for every dependency — frontend framework, backend, database, auth, deployment, and any key libraries.”

4. FRONTEND_GUIDELINES.md — Lock the Design System

Colors, spacing, typography, component patterns—all of it. Without this, you will see a different button style on each page. This file contains your design contract.

Prompt to create it:

“Create a FRONTEND_GUIDELINES.md for my app. Include: color palette (hex values), typography scale, spacing system, button variants, form styles, and any reusable component patterns I should follow consistently.”

5. `BACKEND_STRUCTURE.md` — Schema and Endpoints

All database tables. Each API endpoint. What it accepts determines what it returns. This is the file that prevents the AI from creating a schema on the fly that conflicts with your frontend.

Prompt to create it:

“Write a `BACKEND_STRUCTURE.md` for this app: [brief description]. Include all database tables with field names and types, relationships between tables, and a list of API endpoints with their methods, inputs, and expected responses.”

6. IMPLEMENTATION_PLAN.md — The Build Sequence

Divide the build into four numbered steps: 1.1, 1.2, 2.1, and 2.2. This is the actual execution order. It prevents the AI from moving on to features that rely on things that have not yet been built.

Prompt to create it:

“Write an IMPLEMENTATION_PLAN.md for this app based on the PRD, app flow, and backend structure. Break it into phases and numbered steps in the correct build order. Start with the foundation and end with polish.”

7. TODO.md — The AI's Execution Tracker (Bonus)

This file isn't just for you — it's for the AI. It's a living checklist the AI reads at the start of every session to know exactly where things stand. Completed steps are marked, pending steps are clear, and nothing gets repeated or skipped.

The power is in how you use it. Instead of re-

explaining context every session, you just say: “*Read TODO.md and execute Step 3.*” The AI picks up exactly where you left off.

After each step is done, tell the AI to mark it complete in the file. Your next session starts with a single line.

Prompt to create it:

“Using the IMPLEMENTATION_PLAN.md, create a TODO.md formatted as a step-by-step checklist the AI will use to track execution progress. Each step should have: a checkbox ([]), a step number, a short action label, and a status (pending/in progress/done). The AI should update this file after completing each step so it always reflects the current state of the build.”

How to Use These Files

Once all six (or seven) files exist, start every AI conversation like this:

“Read todo.md and execute step 1.

That’s it. The AI now has context. It knows the boundaries. It stops inventing things.

Conclusion

The key is to talk it out with AI about what you need and what’s correct and what else can be added. It could take an hour or so. But they save you days of debugging misaligned code, redesigning screens that didn’t match the vision, and re-explaining what you actually wanted.

Write the docs. Then vibe code like you actually know what you’re building.

Want to understand how to build the complete MVP, check this [article](#).

Want to see this in action, checkout this [github repo](#).

Have a system that works better? Drop it in the comments — always curious what other builders are doing.

AI

Vibe Coding

Tips And Tricks

MVP



Published in Artificial Intelligence in Plain English

Follow

42K followers · Last published 1 hour ago

New AI, ML and Data Science articles every day.
Follow to join our 3.5M+ monthly readers.



Written by Ruqaiya Beguwala

Follow

292 followers · 46 following

Software Engineer | Writer | Hustler

No responses yet



Pancake Agree

What are your thoughts?

More from the list: "Implement"

Curated by Pancake Agree



Mark Chen

**I Used Claude Code to
Build a Personal...**

★ · Apr 6



In Vibe Cod... by Alex Dun...

**The AI Drop That
Changes Everything.**

★ · 4d ago



[View list](#)

Recommended from Medium



In Vibe Coding by Alex Dunlop

**I Cut Claude Code’s
Output Tokens by 75%...**

Fewer tokens. Somehow more accurate. There’s a paper.



6d ago



992



12



In The Ai Studio by Ai studio

**What Is Andrej
Karpathy’s CLAUDE....**

And Why 10,000+ Developers Downloaded It



Apr 7



218



1



In AI Advances by Yanli Liu

I Ran Codex and Claude Side by Side. Here's...

The setup takes 5 minutes.
The compliance question...

★ 5d ago 🖱️ 424 💬 3 💬

Kristopher Dunham

MemPalace: The Viral AI Memory System...

Every AI agent you've ever built has the same fatal flaw.

...★ Apr 8 🖱️ 604 💬 8 💬 ...

In UX Planet by Nick Babich

CLAUDE.md Best Practices

10 Sections to Include in your CLAUDE.md

★ Mar 6 🖱️ 657 💬 10 💬

In Towards AI by Rick Hightower

Claude Code 2026: The Daily Operating Syste...

The exact 5-part model, 10-minute routine, slash...

...★ Apr 7 🖱️ 292 💬 ...

See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Rules](#) [Terms](#)
[Text to speech](#)